SISTEMA DE GESTIÓ EMPRESARIAL

UD01. INTRODUCCIÓ A DOCKER

Activitats Avaluables

Miriam Mendieta Racedo.-
2DAMS
20/10/2025

ÍNDEX

INTRODUCCIÓ 1

ACTIVITAT 01 2
ACTIVIDAD 02 11
ACTIVITAT 03 14
ACTIVITAT 04 19
ACTIVITAT 05 22

BIBLIOGRAFIA 23

INTRODUCCIÓ

En el món del desenvolupament de programari, hi ha un desafiament constant i frustrant: la inconsistència de l' entorn. La típica frase " a la meva màquina funciona " ha torçat tot equips de desenvolupament que es preï... on una aplicació perfecta es desmorona en moure' s de l' ordinador del desenvolupador al servidor de proves o a producció.

En aquest context la necessitat d' entorns consistents, portables i fàcilment replicables ha impulsat l' ús de tecnologies com **Docker**. Aquesta eina ofereix un enfocament simple però poderós per empaquetar una aplicació i totes les seves dependències —biblioteques, arxius de configuració i fins i tot l'entorn operatiu— en una unitat estandarditzada anomenada Imatge. Aquesta Imatge actua com un pla immutable i el **Contenidor**, instància viva, lleugera i aïllada, que garanteix que l' entorn d' execució sigui idèntic en qualsevol lloc.

Al llarg d' aquesta sèrie d' exercicis, s' explora els fonaments de Docker i la seva aplicació pràctica en diferents tipus d' escenaris.

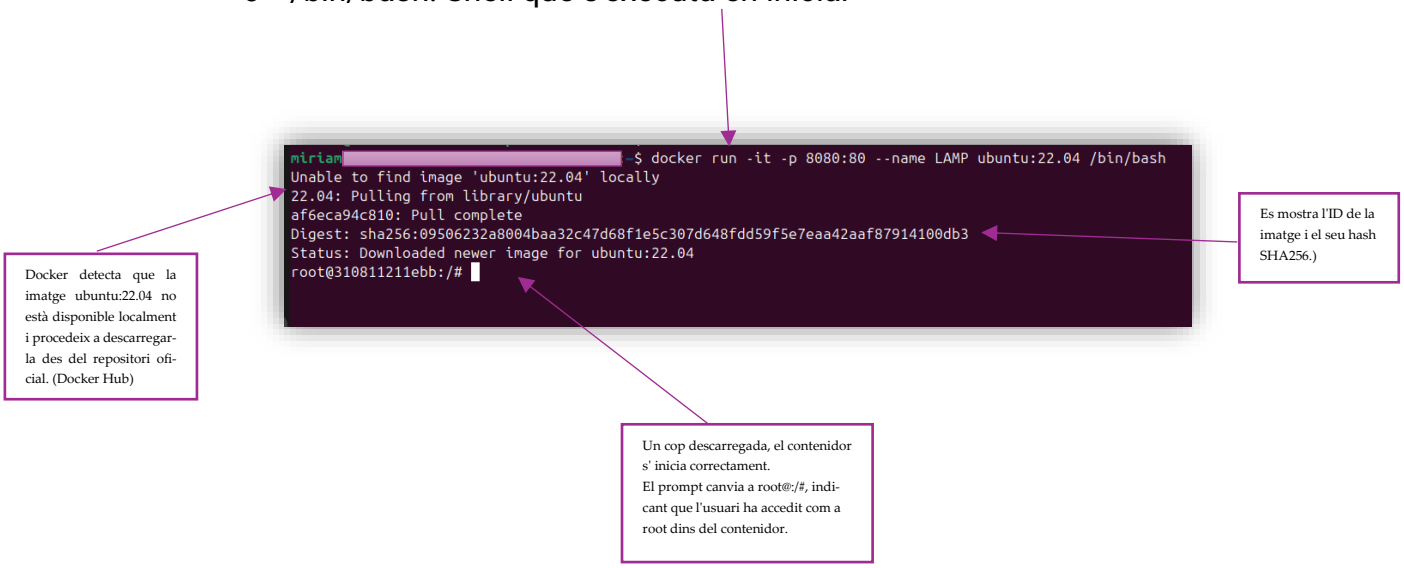
ACTIVITAT 01

Cas pràctic 02 - Instal·lant LAMP + Wordpress en contenidor

1. Inici de contenidor Docker per a entorn LAMP

Es treballa des d'una terminal Ubuntu, on s'executa el comandament per iniciar un contenidor Docker amb Ubuntu 22.04, destinat a configurar un entorn LAMP (Linux, Apache, MySQL/MariaDB, PHP).

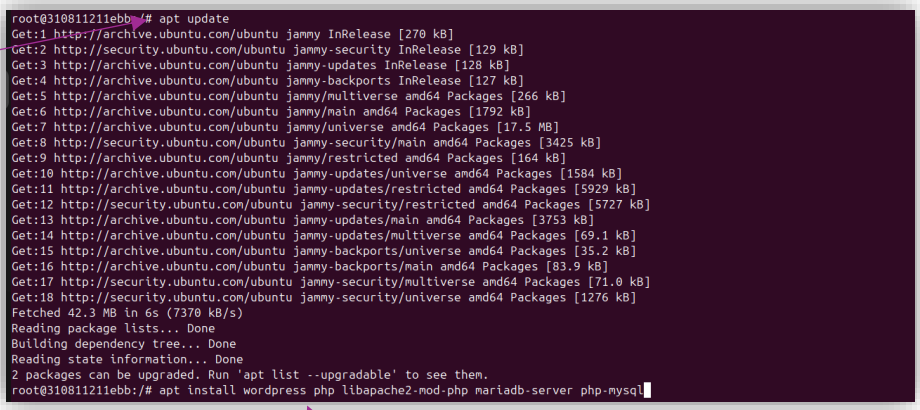
1. -it: Mode interactiu amb terminal
2. -p 8080:80: Redirigeix el port 8080 del host al 80 del contenidor
3. --name LAMP: Assigna el nom "LAMP" al contenidor
4. ubuntu:22.04: Imatge base utilitzada
   - /bin/bash: Shell que s'executa en iniciar



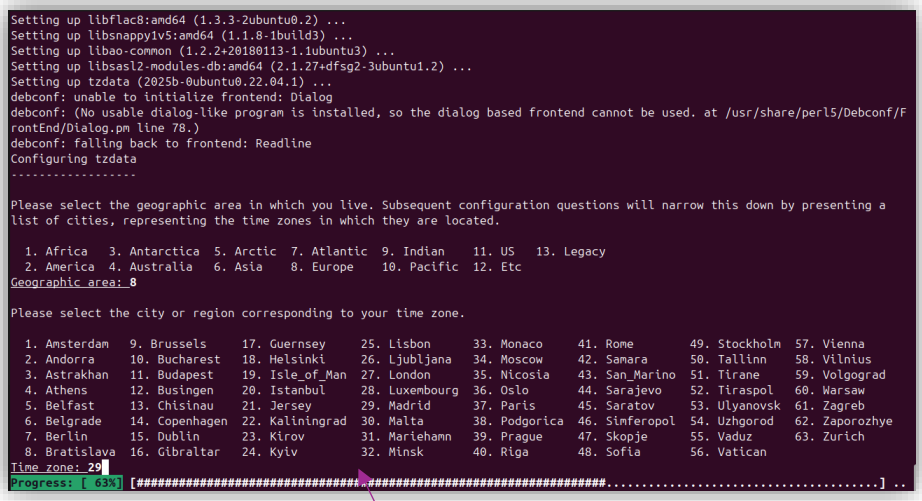
1. Instal·lant LAMP I Wordpress

Es prepararà l'entorn d'execució de WordPress dins del contenidor per garantir que Apache pugui servir contingut dinàmic i s'estableix la connexió entre WordPress i la base de dades MariaDB.

Actualització de repositoris



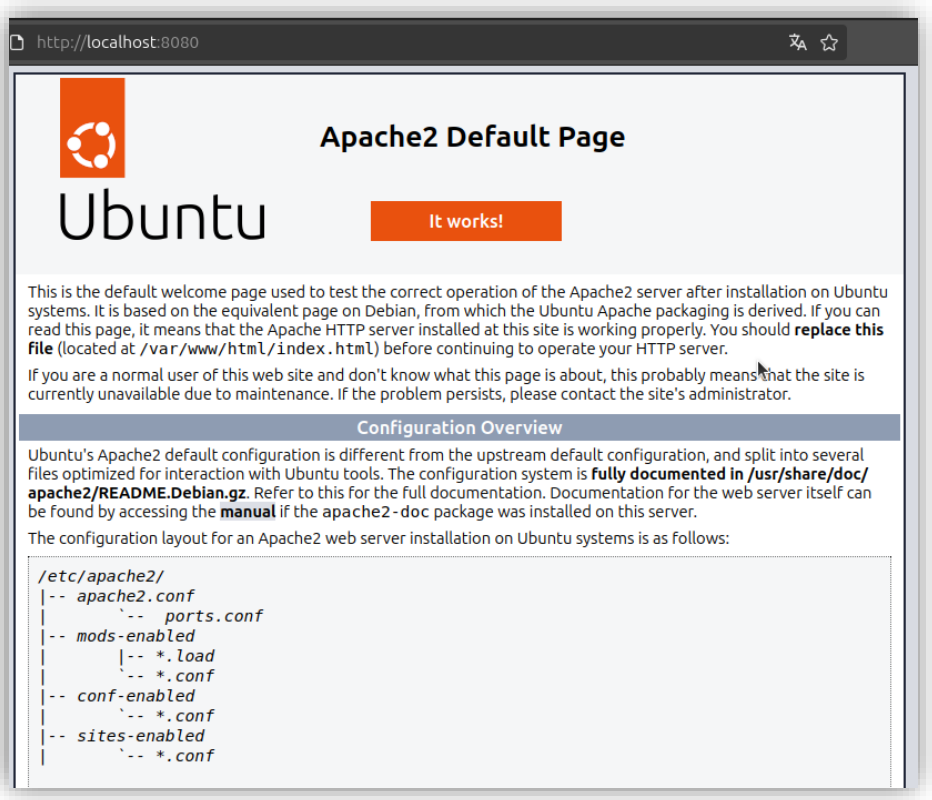
1. apt install: gestor de paquet que permet la instal·lació.
2. wordpress: CMS que s'instal·larà a /usr/share/wordpress.
3. php: Llenguatge de programació per al backend.
4. libapache2-mod-php: Mòdul que permet a Apache interpretar PHP.
5. mariadb-server: Motor de base de dades compatible amb MySQL.
6. php-mysql: Extensió que permet a PHP interactuar amb MariaDB.



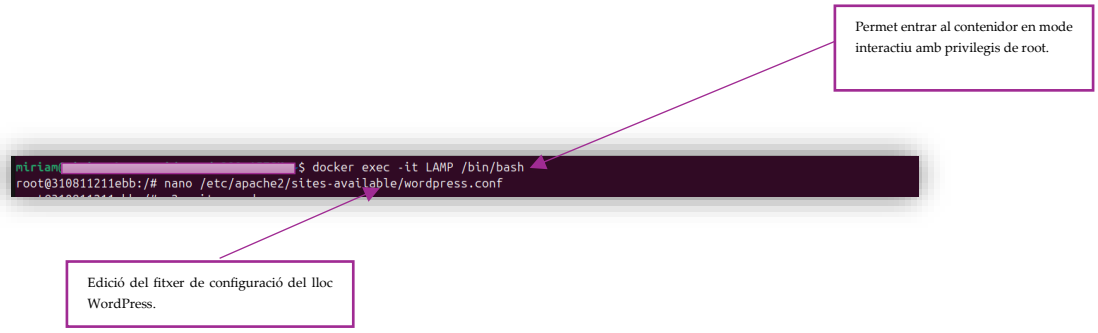
El sistema demana a l'usuari seleccionar una zona geogràfica. En aquest cas, s' elegeix l' opció 8, corresponent a Europa.
Després es presenta una llista de ciutats europees. L'usuari selecciona l'opció 29 (Madrid).
Assegura que els registres d'Apache, MariaDB i WordPress tinguin l'hora correcta.

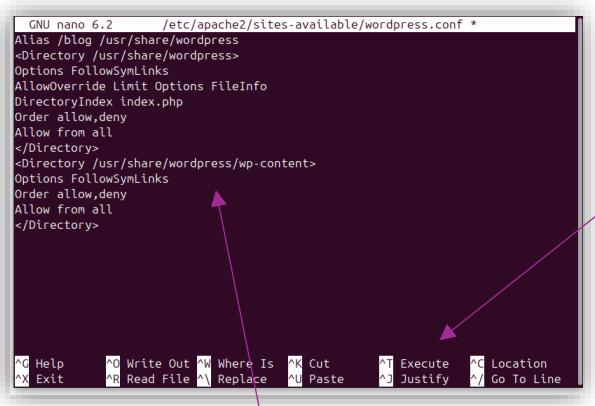
2. Es llança el servei Apache amb el comandament: ***service apache2 start***

I es comprova : ***http://localhost:8080***



❖ Preparant Apache para usar Wordpress

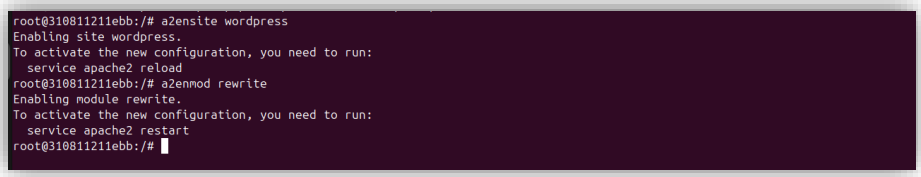




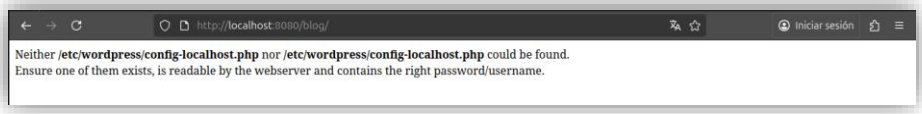
Guarda i tanca Nano
Ctrl + O → Guardar
Enter → Confirmar nom
Ctrl + X → Sortir

Alias /blog /usr/share/wordpress	Permite acceder a WordPress desde http://localhost:8080/blog
<Directory /usr/share/wordpress>	Define permisos y comportamiento del directorio principal
Options FollowSymLinks	Permite seguir enlaces simbólicos (útil para plugins y temas)
AllowOverride Limit Options FileInfo	Permite que .htaccess controle ciertas directivas
DirectoryIndex index.php	Define el archivo principal que se carga por defecto
Order allow,deny + Allow from all	Permite acceso público al directorio

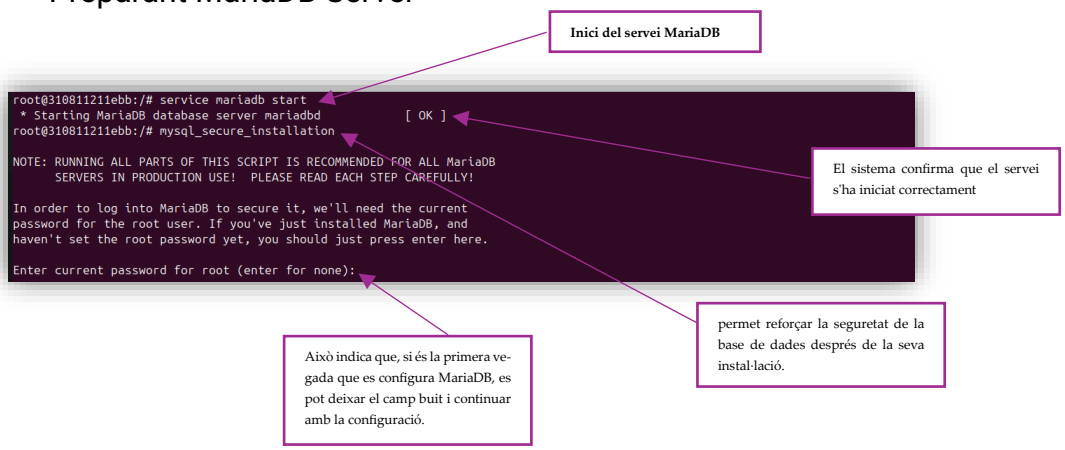
Carregar el lloc, habilitar "URL rewriting" i recarregar Apache:

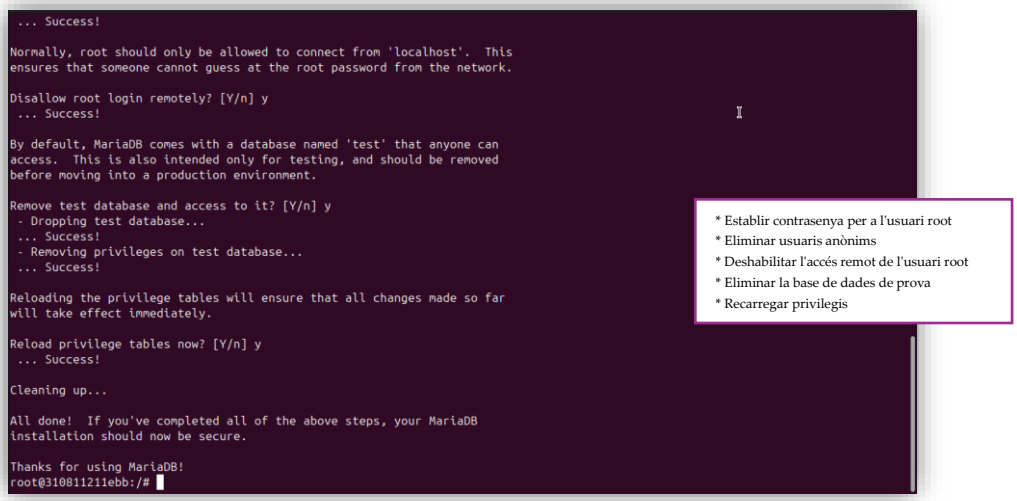


Comprovació amb la URL http://localhost:8080/blog, tot i que es veu un error perquè encara no hem configurat el fitxer "/etc/wordpress/config-localhost.php"

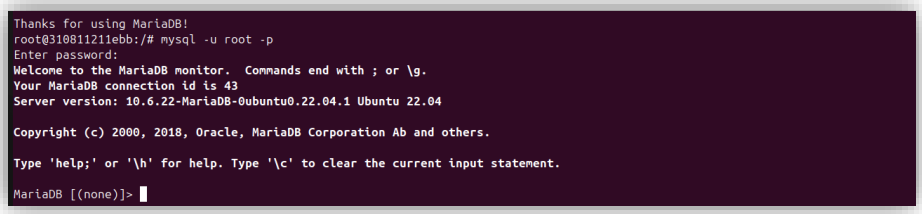


- Preparant MariaDB Server

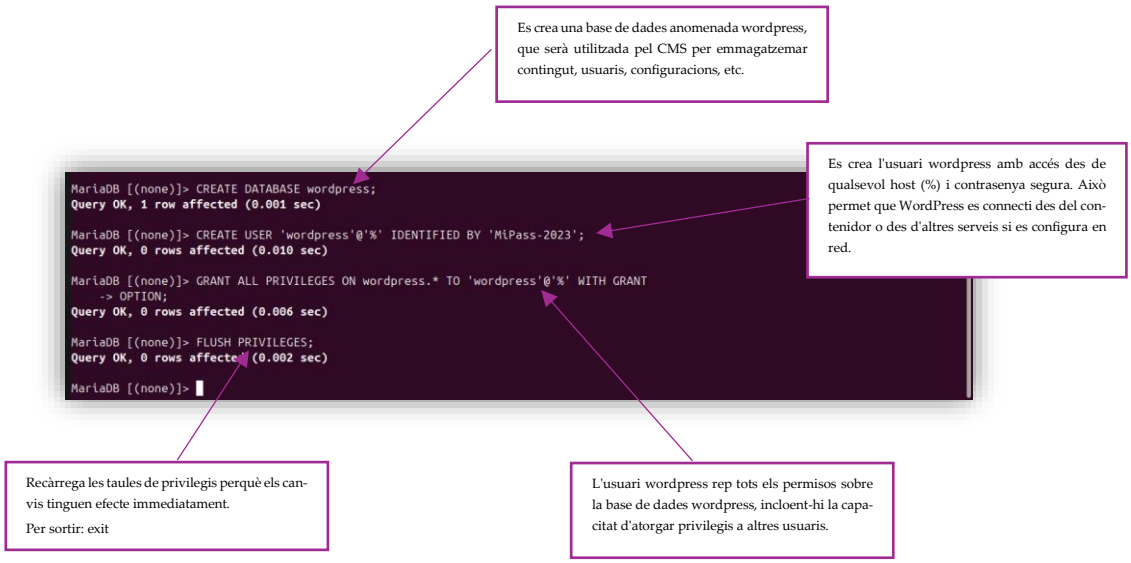




S' accedeix a la base de dades amb el client MySQL de la manera següent:

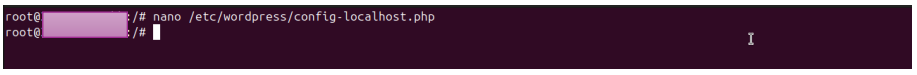


Creació de base de dades i usuari per a WordPress a MariaDB



❖ Configurant Wordpress

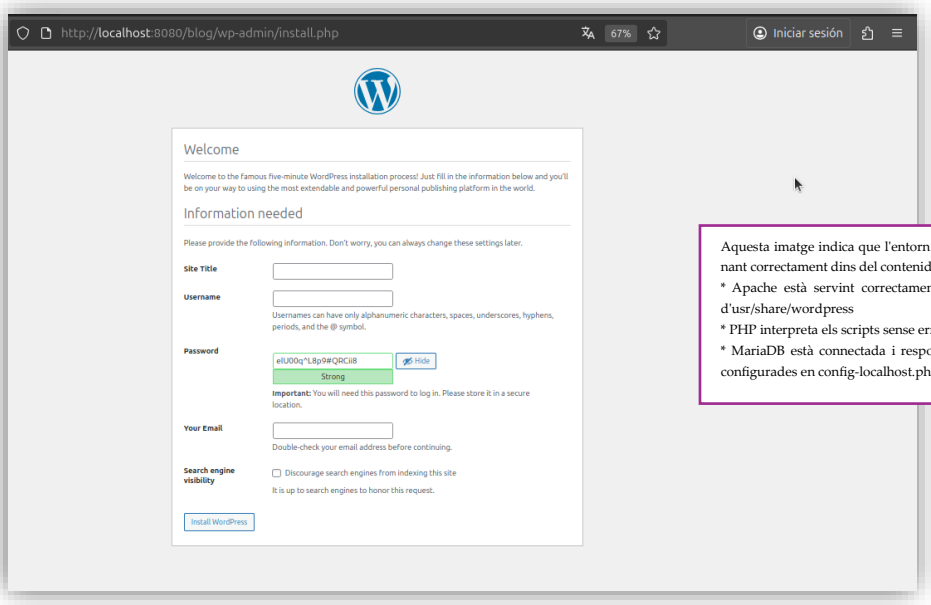
Crear (si no existeix) o editar el fitxer de configuració de Wordpress:





DB_NAME	Nombre de la base de datos que usará WordPress (wordpress)
DB_USER	Usuario con permisos sobre la base de datos (admin)
DB_PASSWORD	Contraseña del usuario (MiPass-2023)
DB_HOST	Dirección del servidor de base de datos (localhost)
DB_CHARSET	Codificación de caracteres (utf8)
DB_COLLATE	Reglas de ordenamiento y comparación (utf8_general_ci)
WP_CONTENT_DIR	Ruta al directorio de contenido de WordPress

S' accedeix a la URL **http://localhost:8080/blog**, on es pot configurar el lloc Wordpress, indicant nom de lloc, usuari i email.



Aquesta imatge indica que l'entorn LAMP està funcionant correctament dins del contenidor Docker en xarxa:
* Apache està servint correctament el contingut des d'usr/share/wordpress
* PHP interpreta els scripts sense errors
* MariaDB està connectada i respon a les credencials configurades en config-localhost.php

❖ Configurant els serveis perquè arrenquin en iniciar el contenidor

```
miriam                          :~$ docker exec -it LAMP /bin/bash
root@310811211ebb:/# cd /root
root@310811211ebb:~# nano .bashrc
root@310811211ebb:~# docker stop LAMP
```

Permet afegir comandaments que s'executaran automàticament en iniciar bash...

Parar el contenedor

```
 GNU nano 6.2                                  .bashrc *
    alias grep='grep --color=auto'
    alias fgrep='fgrep --color=auto'
    alias egrep='egrep --color=auto'
fi

# some more ls aliases
alias ll='ls -alF'
alias la='ls -A'
alias l='ls -CF'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
    . ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
#if [ -f /etc/bash_completion ] && ! shopt -oq posix; then
#    . /etc/bash_completion
#fi
service apache2 start
service mariadb start

^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```

S'ha d'anar al final de l'arxiu i col·locar aquests 2 comandaments que permeten l'arrencada dels serveis des de l'inici

```
miriam                          ~$ docker start LAMP
LAMP
miriam                          ~$
```

Inicia un contenidor prèviament creat i detingut.

La terminal respon amb LAMP, confirmant que el contenidor s'ha iniciat correctament.

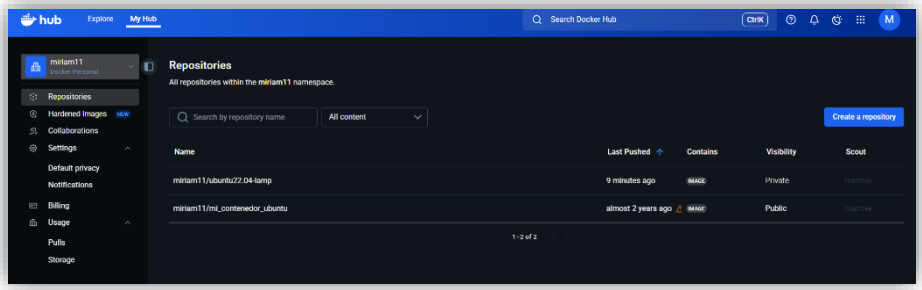
❖ Pujant el contenidor a Docker Hub

```
miriam                          :~$ docker tag ubuntu:22.04 miriam11/ubuntu22.04-lamp
miriam                          :~$ docker info
Client: Docker Engine - Community
 Version:    28.4.0
 Context:    default
 Debug Mode: false
 Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.28.0
    Path:     /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version:  v2.39.4
    Path:     /usr/libexec/docker/cli-plugins/docker-compose

Server:
 Containers: 3
  Running: 0
  Paused: 0
  Stopped: 3
 Images: 2
 Server Version: 28.4.0
 Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Using metacopy: false
  Native Overlay Diff: true
  userxattr: false
 Logging Driver: json-file
 Cgroup Driver: systemd
 Cgroup Version: 2
 Plugins:
```

```
 containerd version: 05044ec0a9a75232cad458027ca83437aae3f4da
 runc version: v1.2.5-0-g59923ef
 init version: de40ad0
 Security Options:
  apparmor
  seccomp
   Profile: builtin
  cgroupns
 Kernel Version: 6.14.0-29-generic
 Operating System: Ubuntu 24.04.3 LTS
 OSType: linux
 Architecture: x86_64
 CPUs: 4
 Total Memory: 7.544GiB
 Name: miriam-Lenovo-ideapad-320-15ISK
 ID: 44955c65-5849-47c5-ab1a-b4b07ff30909
 Docker Root Dir: /var/lib/docker
 Debug Mode: false
 Username: miriam11
 Experimental: false
 Insecure Registries:
  ::1/128
  127.0.0.0/8
 Live Restore Enabled: false

miriam@miriam-Lenovo-ideapad-320-15ISK:~$ docker push miriam11/ubuntu22.04-lamp
Using default tag: latest
The push refers to repository [docker.io/miriam11/ubuntu22.04-lamp]
767e56ba346a: Mounted from library/ubuntu
latest: digest: sha256:89eecb4e2b529c4550b4fb475c2de8c3a19e63697265714aaffe9f443c221187 size: 529
miriam@miriam-Lenovo-ideapad-320-15ISK:~$
```



UD 03. Cas pràctic 03 - Accedint a interfície gràfica amb NoVNC

1. Preparació del contenidor

```
miriam                                :~$ docker run --rm -it -p 8080:8080 theasp/novnc
Unable to find image 'theasp/novnc:latest' locally
latest: Pulling from theasp/novnc
10ffc47270cd: Pull complete
2b363779fc2a: Pull complete
dc199bc6b323: Pull complete
Digest: sha256:244a8e4fd7b49532b1b264764e11af2f6ea2801c725bf09adcdaed155ca1cfa0
Status: Downloaded newer image for theasp/novnc:latest
+ RUN_FLUXBOX=yes
+ RUN_XTERM=yes
+ case $RUN_FLUXBOX in
+ case $RUN_XTERM in
+ exec supervisord -c /app/supervisord.conf
```

- --rm → Elimina el contenidor en tancar-lo (ideal per a proves)
- -it → Mode interactiu amb terminal
- -p 8080:8080 → Exposa el port 8080 del contenidor al host (per accedir via navegador)
- theasp/novnc → Imatge que ja inclou servidor VNC + client NoVNC llest per usar

Accedir a la interfície gràfica

- Un cop el contenidor estigui corrent:
- Obrir el navegador.
- Anar a http://localhost:8080/vnc.html
- Es carregarà el client **NoVNC** (HTML5 + JavaScript).
- Fer clic a **Connect** per iniciar la sessió VNC.
- Interfície gràfica dins del contenidor.





La imatge theasp/novnc sol incloure un entorn gràfic lleuger com XFCE o similar, ideal per a proves. Es pot interactuar amb ell com si fos un escriptori remot.

En tancar la terminal, el contenidor s'elimina (pel paràmetre --rm), si es vol mantenir-lo actiu, elimina --rm i afegeix -d per a mode daemon:

docker executar -d -p 8080:8080 theasp/novnc

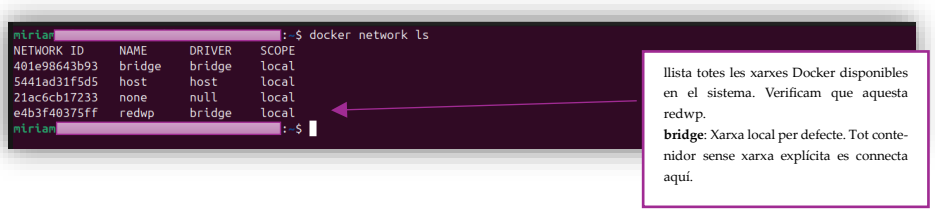
ACTIVIDAD 02

UD 05. Cas pràctic 01- Wordpress + MariaDB

S'ha de posar en marxa el popular CMS Wordpress. Per a això farem servir una xarxa on es connectaran dos contenidors: el primer utilitzant Apache + PHP, juntament amb una versió instal·lada de Wordpress, mentre que el segon contindrà un servidor de bases de dades MariaDB. A més, farem un exemple d'una migració de versió del contenidor MariaDB.

1. CREACIÓ DE LA XARXA

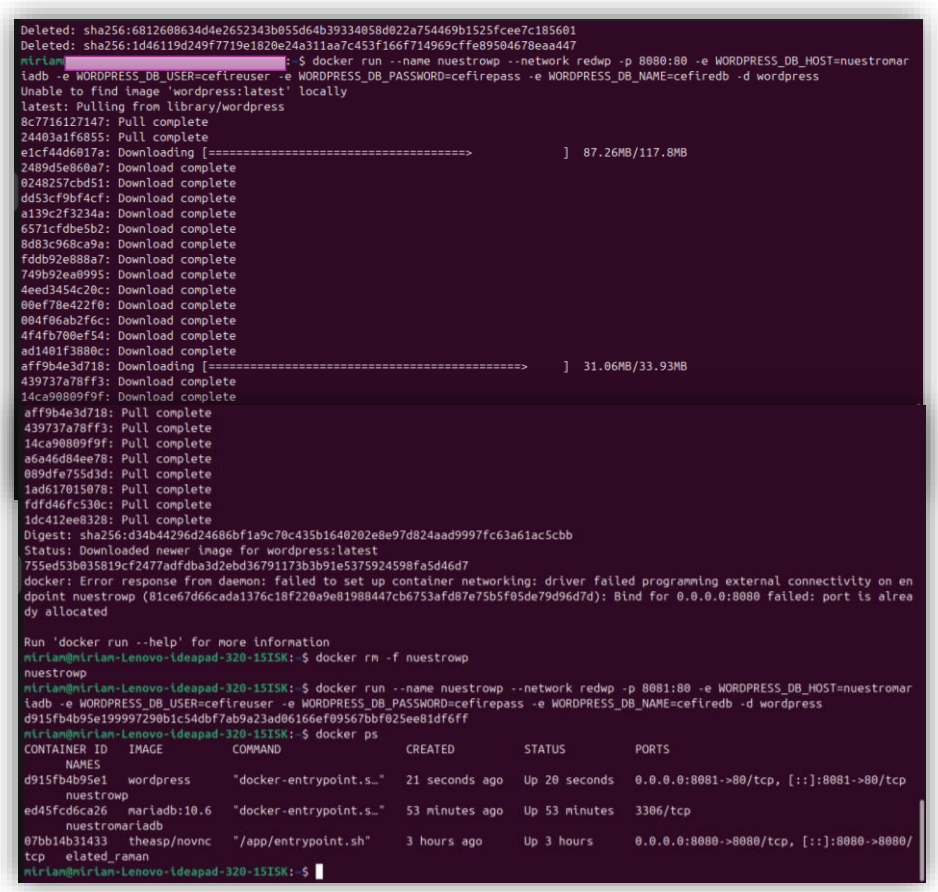




2. CREACIÓ CONTENIDOR MARIADB

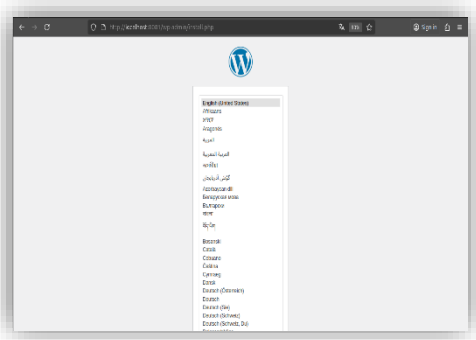


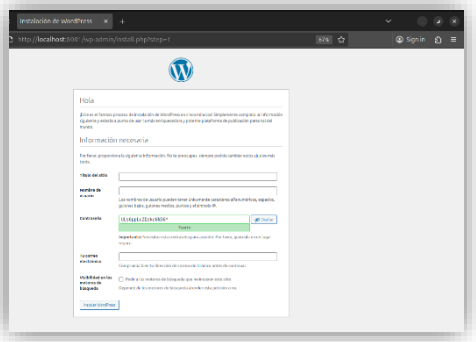
3. CREACIÓ CONTENIDOR AMB WORDPRESS



Estava donant error al port 8080 i es canvia al 8081

4. INSTAL·LACIÓ WORDPRESS

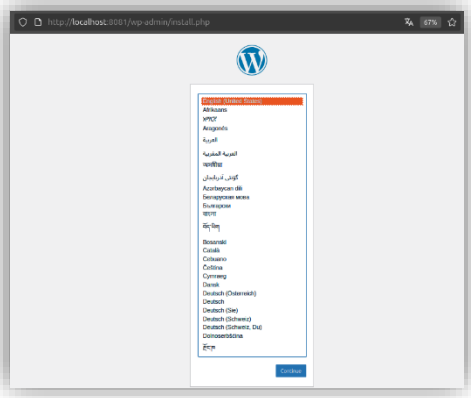


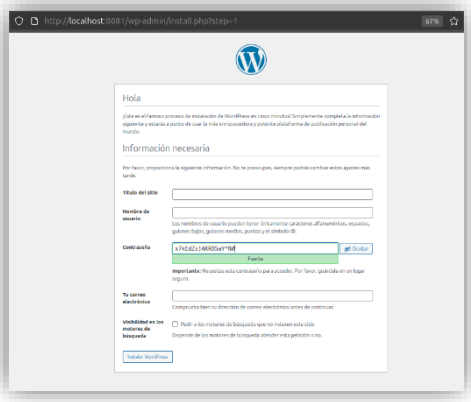


5. MIGRANT CONTENIDOR MARIADB DE 10.6 A 10.7

```
miriam                          :~$ docker stop nuestromariadb
nuestromariadb
miriam                          :~$ docker rm nuestromariadb
nuestromariadb
```

```
miriam                          :~$ docker run --name nuestromariadb --network redwp -v /home/miriam/mariadbdata:/var/lib/
mysql -e MARIADB_ROOT_PASSWORD=cefireroot -e MARIADB_USER=cefireuser -e MARIADB_PASSWORD=cefirepass -e MARIADB_DATABASE=cefiredb
 -d mariadb:10.7
Unable to find image 'mariadb:10.7' locally
10.7: Pulling from library/mariadb
99803d4b97f3: Pull complete
b8bc823a83fd: Pull complete
16685f710f5d: Pull complete
b5660ff63058: Pull complete
8b13582f0741: Pull complete
7d98acc143e6: Pull complete
6e4fbbea63e0: Pull complete
d3da17671559: Pull complete
Digest: sha256:9a48ac9f196f3d4fd6fea2cab59a49df9e7ca459bf14b2f7b85a0e38a5454571
Status: Downloaded newer image for mariadb:10.7
6213257960d5d057f23d90a191922a4e8b8a0aef1fc34ba8dbb40da7d6c842a8
```





ACTIVITAT 03

UD 06. Cas pràctic 01 - Wordpress + MariaDB

Es posarà en marxa el popular CMS Wordpress. Per a això es farà servir un fitxer "docker-compose.yml" comentat que ens posarà en marxa dos contenidors: el primer utilitzant "Apache + PHP", juntament amb una versió instal·lada de Wordpress, mentre que el segon contindrà un servidor de bases de dades MariaDB.

Contenido del fichero “docker-compose.yml”(explicación por línea →)

#Versión del fichero docker-compose 3.9. No obligatorio desde la versión de docker-compose 1.27.0

version: "3.9" # →Define la versión del esquema de docker-compose. Aunque desde la versión 1.27.0 no es obligatorio, incluirlo ayuda a mantener compatibilidad y claridad

#Indicamos los servicios a lanzar

services: # →Aquí comienzan los contenedores que se van a lanzar. Cada uno se define como un servicio independiente.

#Plantill del servicio "db"

db:

#Se basa en la imagen "mariadb", version 10.11.2

image: mariadb:10.11.2 #-->Crea un contenedor basado en la imagen oficial de MariaDB, versión 10.11.2.

#Mapea en el volumen "db_data" el directorio "/var/lib/mysql", lo que da persistencia al contenido de

#Wordpress almacenado en la base de datos

volumes:

- **db_data:/var/lib/mysql** #-->Mapea el volumen db_data al directorio interno donde MariaDB guarda los datos. Esto garantiza persistencia: si el contenedor se borra, los datos siguen ahí.

#Indica que siempre que el servicio finalice, se reiniciará

restart: always #-->Si el contenedor se detiene por error, se reinicia automáticamente. Muy útil en entornos de producción o pruebas continuas.

#Define un conjunto de variables de entorno para estos contenedores,

#indicando password de root de mariadb, nombre de base de datos,

usuario con permisos root (necesario para conexiones remotas) y password de ese usuario

environment: #-->Variables de entorno que configuran la base de datos

MARIADB_ROOT_PASSWORD: somewordpress #-->clave del superusuario

MARIADB_DATABASE: wordpress #-->nombre de la base de datos

MARIADB_USER: wordpress

MARIADB_PASSWORD: wordpress #-->usuario con permisos para acceder desde WordPress.

#Plantilla del servicio "wordpress"

wordpress:

#Indicamos que para lanzar este servicio, debe estar en marcha "db"

depends_on:

- db #-->Indica que este servicio depende de que db esté activo. No garantiza que esté listo, pero sí que se inicia antes.

#Indicamos que basa en la imagen "wordpress", version "latest"

image: wordpress:latest #-->Usa la última versión de WordPress

#Indicamos que el puerto 80 del contenedor se mapea con el puerto 8000 del anfitrion

ports:

- "8000:80" #-->Mapea el puerto 80 del contenedor (donde WordPress sirve contenido) al puerto 8000 del host. Así puedes acceder desde http://localhost:8000.

#Indica que siempre que el servicio finalice, se reiniciará

restart: always #-->se reinicia automáticamente si falla.

#Definimos “variables de entorno”. Definimos donde conectarnos a la base de datos,

#usuario de la base de datos, password de la base de datos y nombre de la base de datos

environment:

WORDPRESS_DB_HOST: db:3306 #-->nombre del servicio db y puerto 3306.

WORDPRESS_DB_USER: wordpress

WORDPRESS_DB_PASSWORD: wordpress

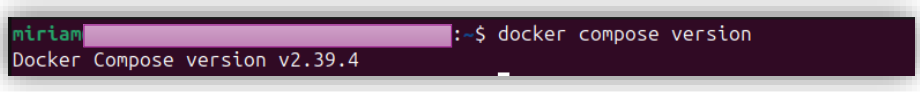
WORDPRESS_DB_NAME: wordpress #-->deben coincidir con los definidos en MariaDB.

#Indicamos los volúmenes creados y compartidos a lo largo del fichero docker-compose.yml

volumes:

db_data: #-->Define el volumen db_data que se usó en el servicio db. Docker lo crea automáticamente si no existe

1. POSAR EN MARXA EL SISTEMA
   - Comprovació Docker Compose instal·lat



- Creando el fichero "docker-compose.yml"

```
miriam                              :~$ mkdir wordpress-docker
miriam                              :~$ nano docker-compose.yml
```

```
 GNU nano 7.2                                  docker-compose.yml *
version: "3.9"

services:
  db:
    image: mariadb:10.11.2
    volumes:
      - db_data:/var/lib/mysql
    restart: always
    environment:
      MARIADB_ROOT_PASSWORD: somewordpress
      MARIADB_DATABASE: wordpress
      MARIADB_USER: wordpress
      MARIADB_PASSWORD: wordpress

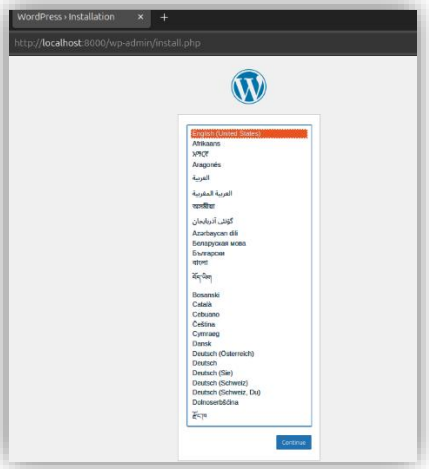
  wordpress:
    depends_on:
      - db
    image: wordpress:latest
    ports:
      - "8000:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: wordpress
      WORDPRESS_DB_NAME: wordpress
```

Claus més importants:

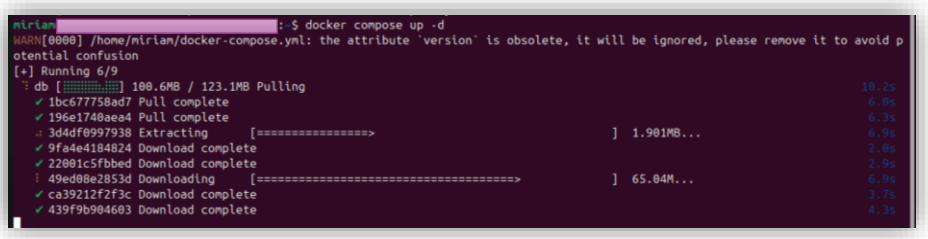
- Indiquem que els servidors de bases de dades ("db")
  - Enllaciten la seva informació a un volum, ddotant-lo de persistència.
  - Defineixin una sèrie de variables d'entorn definint usuaris, contrasenyes i bases de dades a usar.
- Indiquem que els servidors amb Apache + PHP + Wordpress ("wordpress"):
  Per iniciar-se, s'ha d'iniciar abans un servei "db".
  - Estableix variables d' entorn definint valors per a la connexió de base de dades de Wordpress.
  - Enllaça port 80 del contenidor a port 8000 de l'amfitrió.

```
miriam                              :~$ docker compose up -d
WARN[0000] /home/miriam/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid p
otential confusion
[+] Running 9/9
 ✔ db Pulled                                                                                                          19.2s
   ✔ 1bc677758ad7 Pull complete                                                                                        6.0s
   ✔ 196e1740aea4 Pull complete                                                                                        6.3s
   ✔ 3d4df0997938 Pull complete                                                                                        8.4s
   ✔ 9fa4e4184824 Pull complete                                                                                        8.8s
   ✔ 22001c5fbbed Pull complete                                                                                        9.1s
   ✔ 49ed08e2853d Pull complete                                                                                       15.1s
   ✔ ca39212f2f3c Pull complete                                                                                       15.3s
   ✔ 439f9b904603 Pull complete                                                                                       15.5s
[+] Running 4/4
 ✔ Network miriam_default        Created                                                                               0.4s
 ✔ Volume miriam_db_data         Created                                                                               0.2s
 ✔ Container miriam-db-1         Started                                                                               7.4s
 ✔ Container miriam-wordpress-1  Started                                                                               4.3s
```

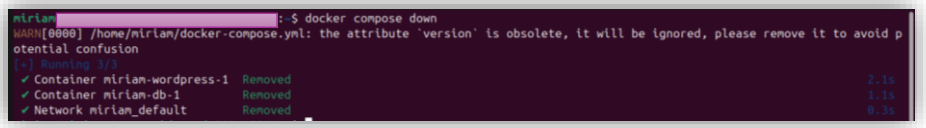
- Provar que tot és correcte accedint a http://localhost:8000

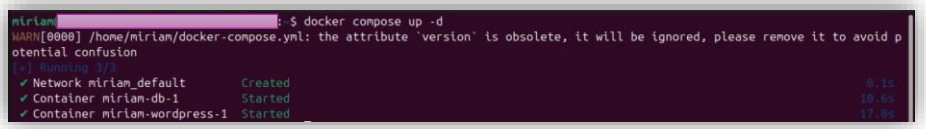


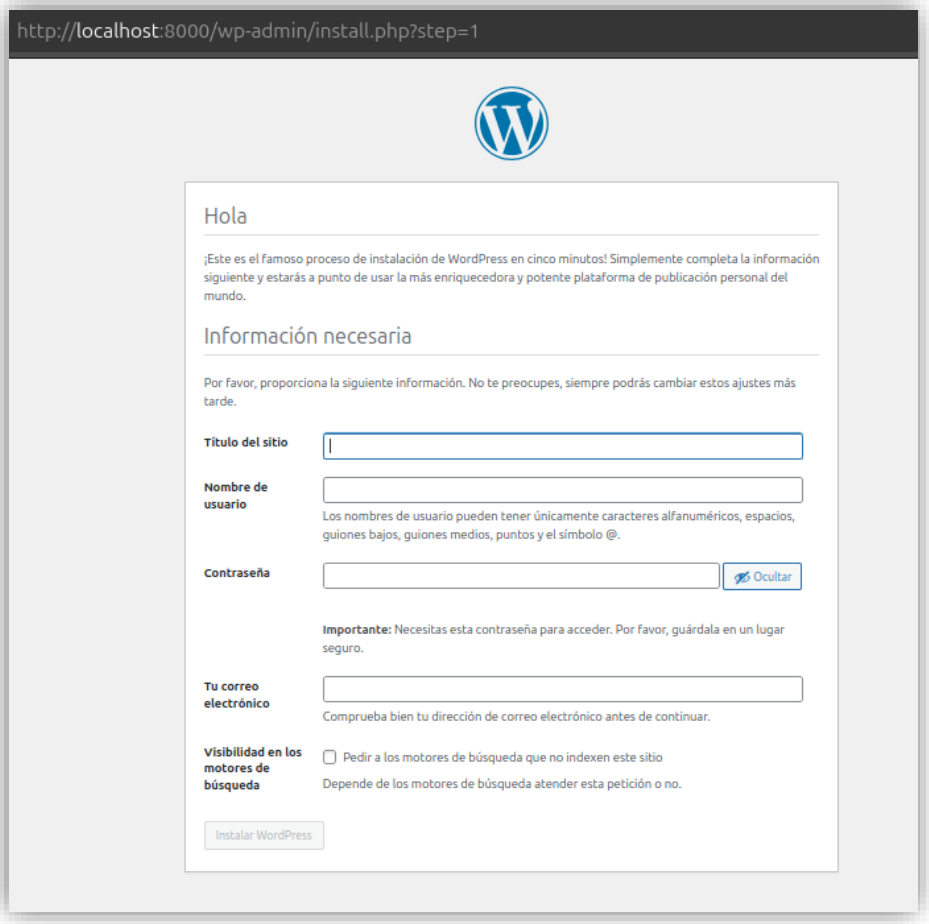
2. PARANT EL SISTEMA



3. RE-LLANÇANT EL SISTEMA







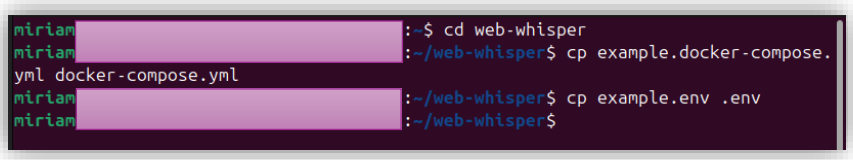
UD 06. Cas pràctic 04 - Whisper AI i Web Whisper

Whisper AI és un programari que permet transcriure àudio a Text. Aquest programari va ser creat per OpenAI (empresa creadora de ChatGPT) i alliberat juntament amb diversos models de processament de llenguatge molt precisos. Actualment, és una de les millors solucions "self-hosted" per convertir àudio a text. Whisper Web és una web, que proporciona una interfície gràfica web per manar àudios a transcriure a Whisper AI. El seu repositori és :

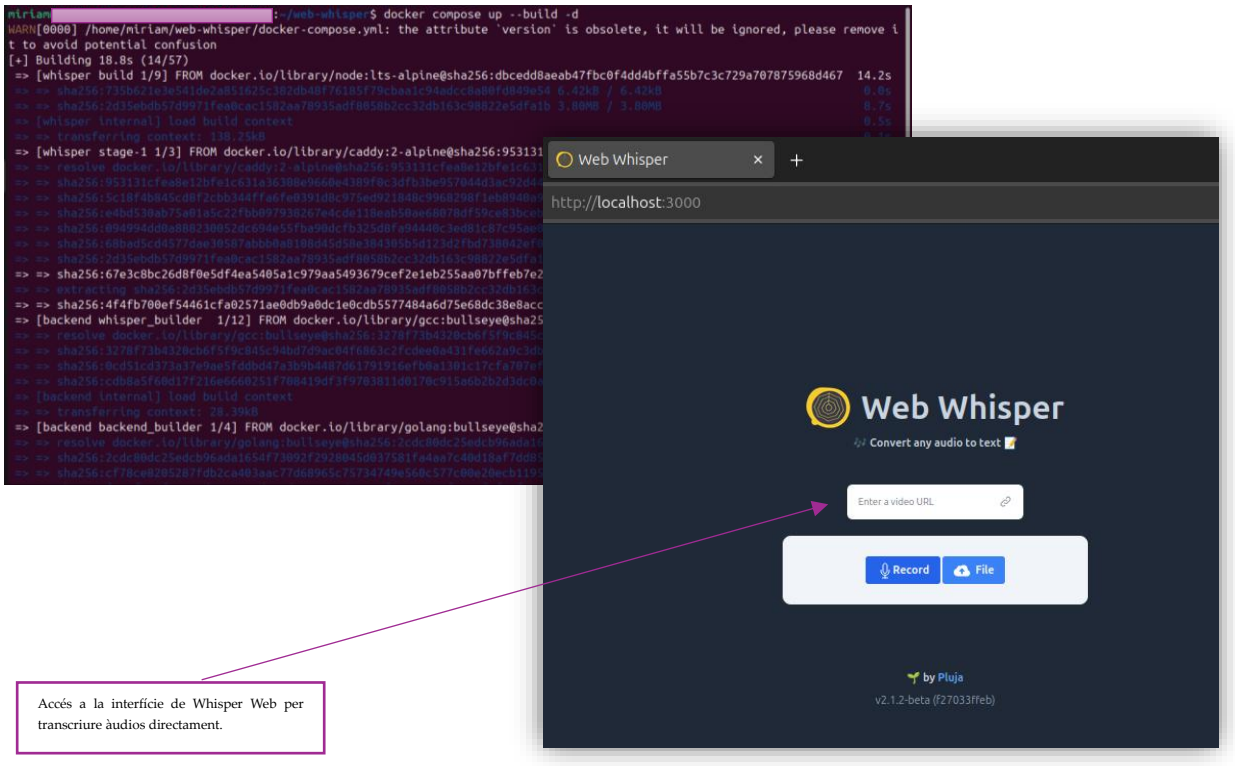
https://codeberg.org/pluja/web-whisper

En aquest cas pràctic vam posar en marxa una versió Dockeritzada extraït del seu repositori seguint les instruccions de https://codeberg.org/pluja/web-whisper/wiki/Self-Hosting

1. Passos previs:
   git clone https://codeberg.org/pluja/web-whisp



2. WHISPER AI Y WHISPER WEB



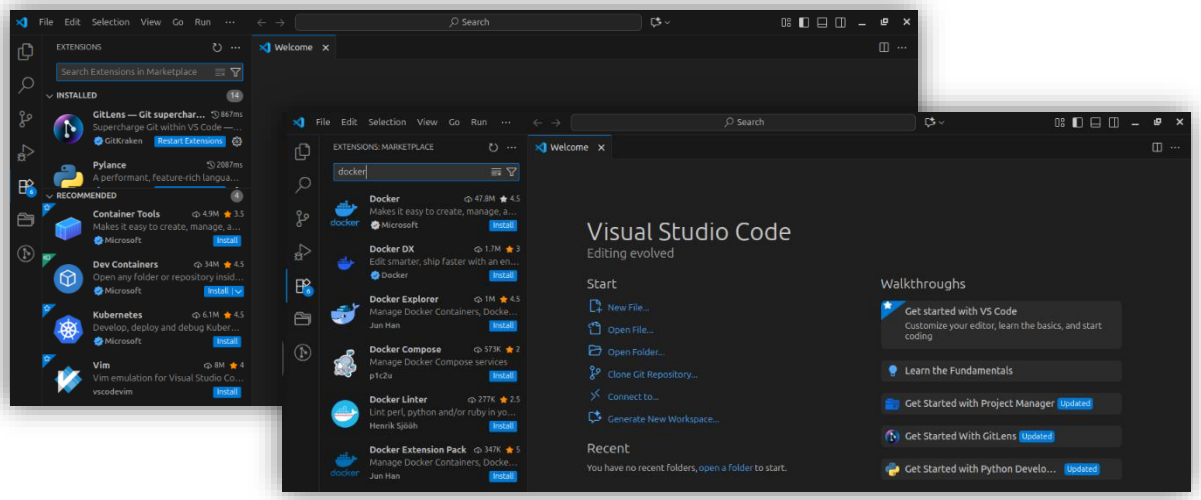
ACTIVITAT 04

UD 07. Cas pràctic 01 - Desenvolupant amb Visual Studio Code en un contenidor

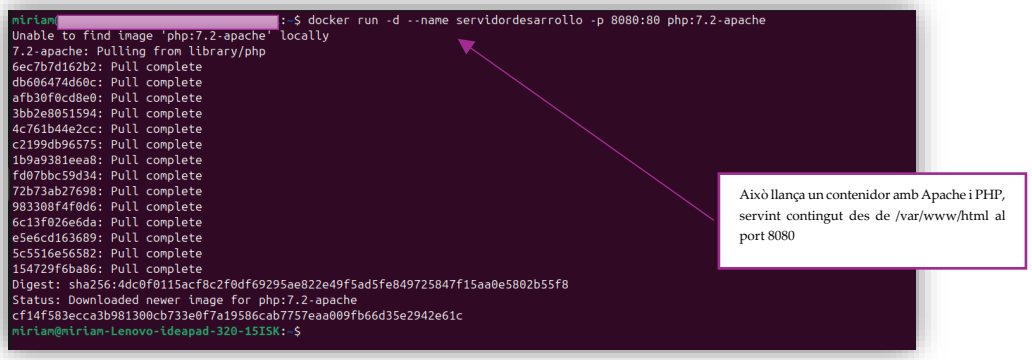
S'utilitzarà l'editor multiplataforma "Visual Studio Code" juntament amb plugins relacionats amb Docker per poder desenvolupar una senzilla aplicació web en un contenidor.

1. Instal·lar eines necessàries
   - Visual Studio Code
     https://code.visualstudio.com/

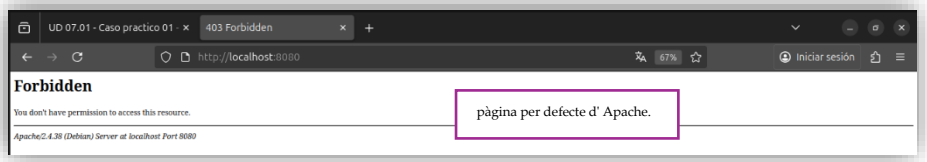
   - Aprèn a usar-lo en
     https://code.visualstudio.com/docs/getstarted/getting-started



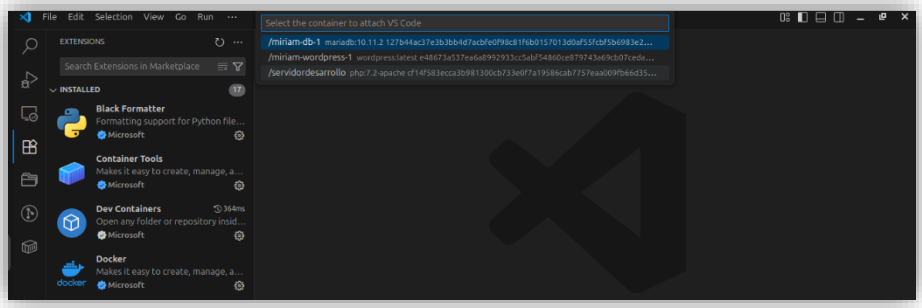
1. CREAR I EDITAR APLICACIÓ EN PHP DINS D' UN CONTENIDOR



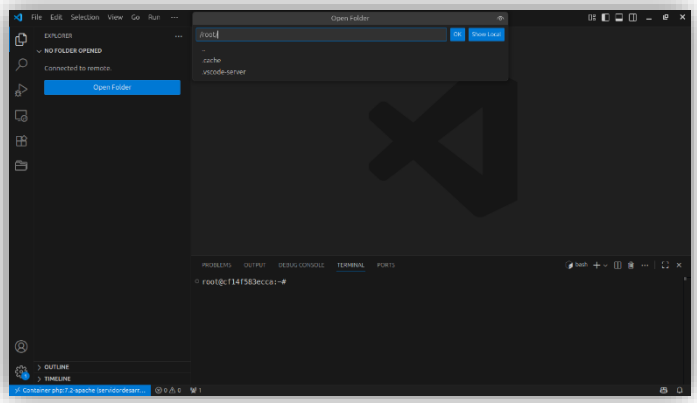
Un cop creat el nostre contenidor provarem d'accedir a http://localhost:8080

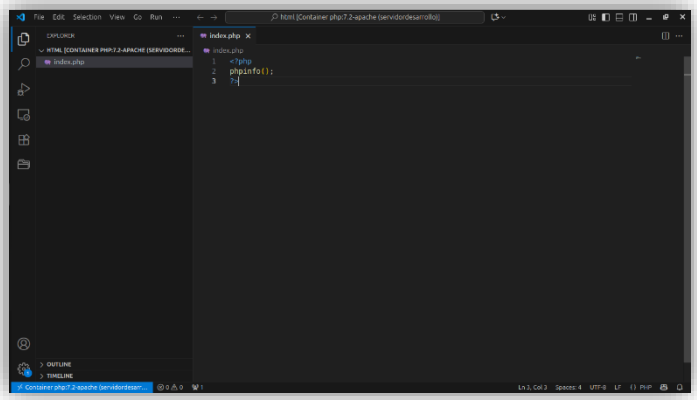


1. Connectar VS Code al contenidor
   - Obrir VS Code
   - Clic a la icona de la ballena(Docker)
   - anar a la pestanya Containers
   - Clic dret sobre servidordesarrollo i selecciona Attach Visual Studio Code o Ctrl+Shift+P →Escribir: *Dev Containers: Attach to Running Container...*
     I seleccionar el contenidor servidordesarrollo



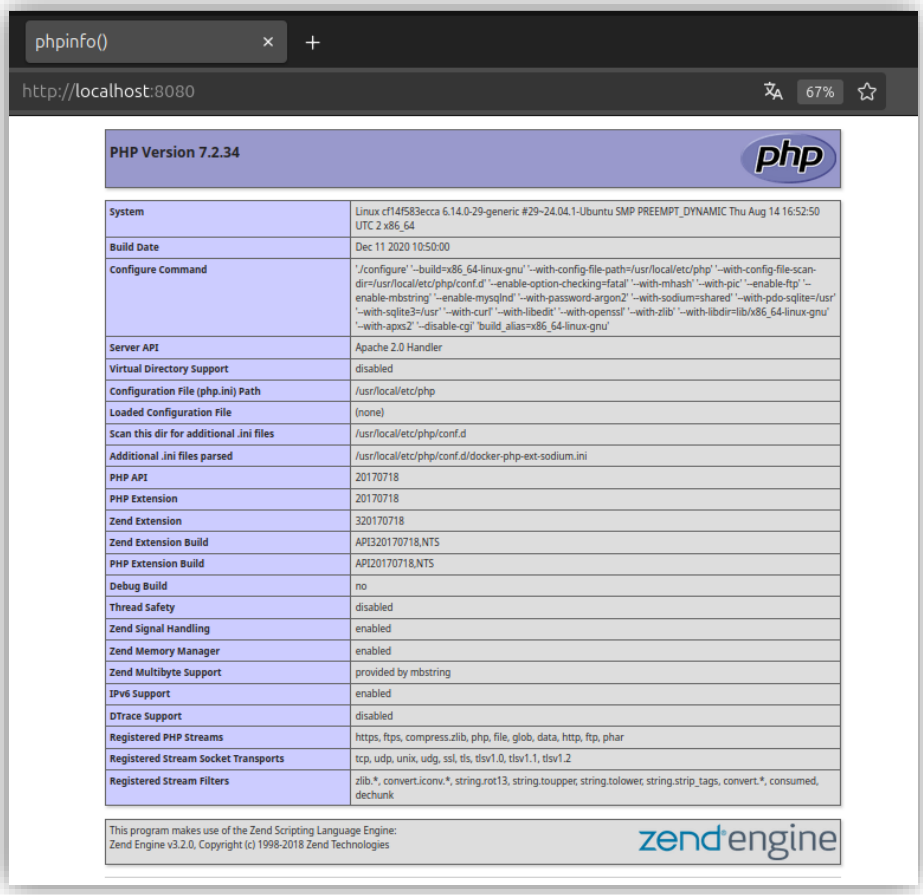
1. Crear arxiu PHP dins del contenidor
   - ➢ Navega al directori /var/www/html
   - ➢ Crea index.php





1. Verificar en navegador

   http://localhost:8080 → veure la sortida de phpinfo() mostrant la configuració del servidor PHP.



phpinfo()

http://localhost:8080

67%

PHP Version 7.2.34

System	Linux cf14f583ecca 6.14.0-29-generic #29~24.04.1-Ubuntu SMP PREEMPT_DYNAMIC Thu Aug 14 16:52:50 UTC 2 x86_64
Build Date	Dec 11 2020 10:50:00
Configure Command	'./configure' '--build=x86_64-linux-gnu' '--with-config-file-path=/usr/local/etc/php' '--with-config-file-scan-dir=/usr/local/etc/php/conf.d' '--enable-option-checking=fatal' '--with-mhash' '--with-pic' '--enable-ftp' '--enable-mbstring' '--enable-mysqlnd' '--with-password-argon2' '--with-sodium=shared' '--with-pdo-sqlite=/usr' '--with-sqlite3=/usr' '--with-curl' '--with-libedit' '--with-openssl' '--with-zlib' '--with-libdir=lib/x86_64-linux-gnu' '--with-apxs2' '--disable-cgi' 'build_alias=x86_64-linux-gnu'
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/usr/local/etc/php
Loaded Configuration File	(none)
Scan this dir for additional .ini files	/usr/local/etc/php/conf.d
Additional .ini files parsed	/usr/local/etc/php/conf.d/docker-php-ext-sodium.ini
PHP API	20170718
PHP Extension	20170718
Zend Extension	320170718
Zend Extension Build	API320170718,NTS
PHP Extension Build	API20170718,NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	enabled
Zend Memory Manager	enabled
Zend Multibyte Support	provided by mbstring
IPv6 Support	enabled
DTrace Support	disabled
Registered PHP Streams	https, ftps, compress.zlib, php, file, glob, data, http, ftp, phar
Registered Stream Socket Transports	tcp, udp, unix, udg, ssl, tls, tlsv1.0, tlsv1.1, tlsv1.2
Registered Stream Filters	zlib.*, convert.iconv.*, string.rot13, string.toupper, string.tolower, string.strip_tags, convert.*, consumed, dechunk

This program makes use of the Zend Scripting Language Engine:
Zend Engine v3.2.0, Copyright (c) 1998-2018 Zend Technologies

ACTIVITAT 05

Llegiu el còmic de "Kubernetes" que està disponible en la següent url: https://cloud.google.com/kubernetes-engine/kubernetescomic?hl=es-419 (o en el PDF Adjunt si falla l'enllaç). Resumir en un màxim de 8 línies que és Kubernetes i perquè raons pot ser útil.

Kubernetes és una plataforma de codi obert per automatitzar el desplegament, escalat i gestió d'aplicacions en contenidors. Organitza les implementacions amb una API de codi obert, agrupant els contenidors en Pods, distribuïts en *nodes* dins d' un *cluster*. Permet escalar serveis fàcilment, reiniciar contenidors fallits, realitzar actualitzacions sense interrupcions i mantenir alta disponibilitat. És útil perquè millora l' eficiència operativa, facilita el desenvolupament continu, i permet gestionar infraestructures complexes de forma declarativa i resilient. Ideal per a entorns cloud, microserveis i aplicacions que requereixen flexibilitat i robustesa.

BIBLIOGRAFIA

- Apunts del Professor.
- https://sergarb1.github.io/CursoIntroduccionADocker
- https://cloud.google.com/kubernetes-engine/kubernetes-comic